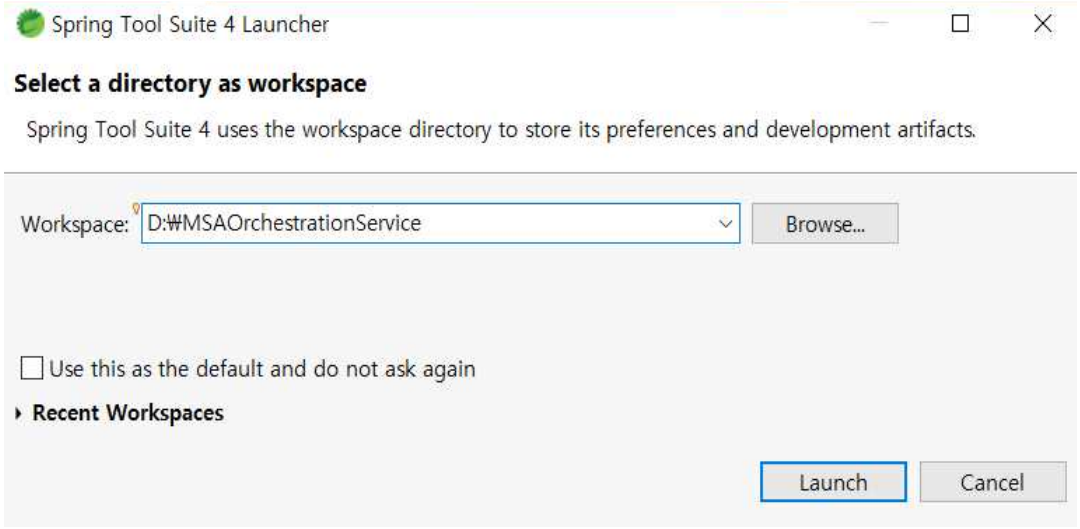
1. 개발 환경 구성 (Windows 기반)

가. 개발 환경 정보

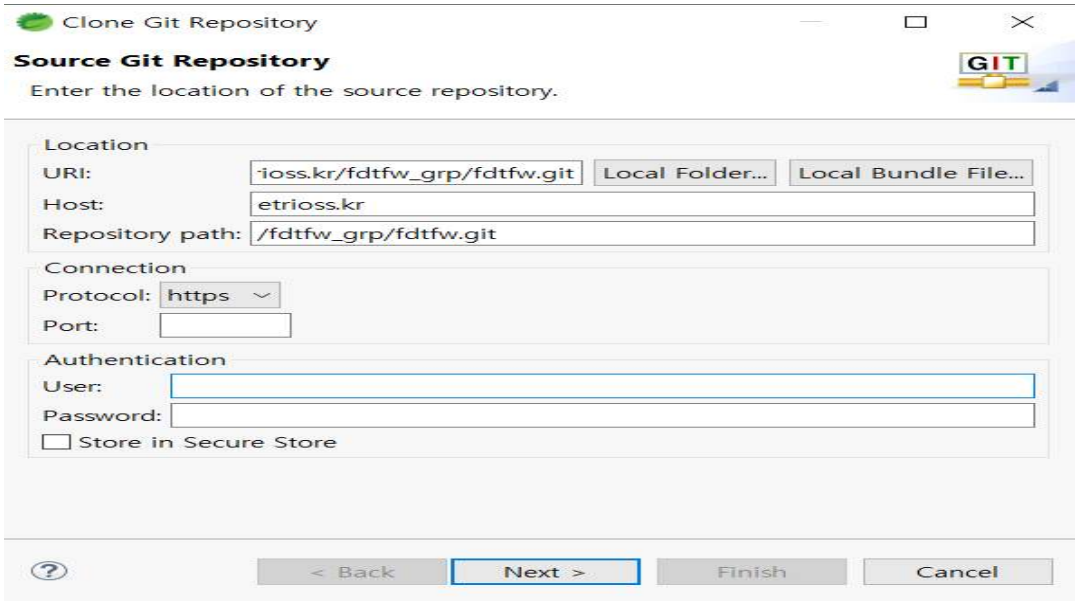
1) 언어 : Java 11
2) 프레임워크 : Spring Boot 2.7.13
3) DB : Mysql 5.7.30
4) 프로젝트 구조 : Spring Boot Maven Multi Module 구조

나. MSA 기반 디바이스 오케스트레이션 설치

1) STS 접속 후 원하는 경로에 WorkSpace 지정



2) Window - Show View - Other - Git - Git Repositories 선택
3) Clone a Git repository 클릭
4) URI 영역에 https://github.com/etri/FeDiT.git 붙여넣기 - Git 계정정보 입력 - Next - main branch 선택



5) 좌측 하단 Git Repositories - Working Tree - OPFW - MSAOC - MSAOrchestrationService 마우스 우클릭 - Import Projects - Show View - Project Expolorer

6) 각각의 모듈 프로젝트 실행

다. DB 세팅

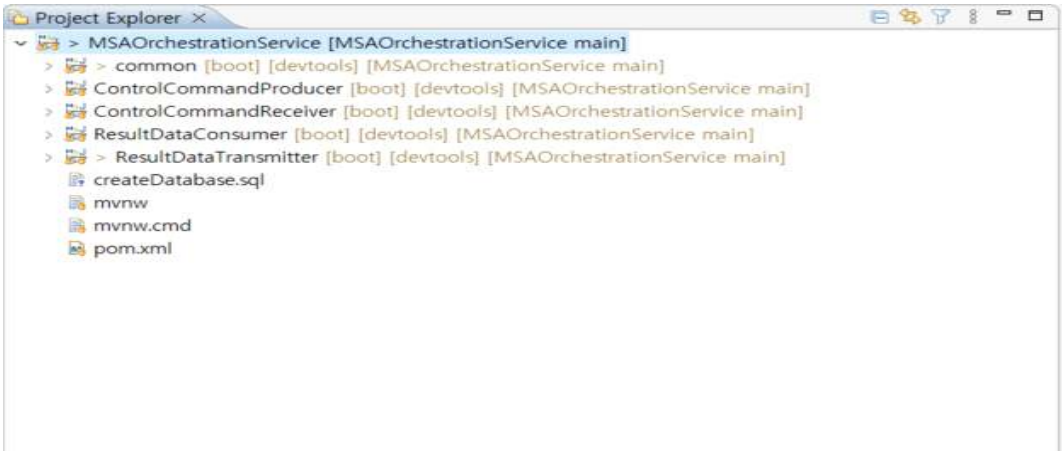
1) Mysql workbench 접속(root 계정으로 접속)

2) 프로젝트 하위에 createDatabase.sql 파일을 로컬 PC에 복사

3) createDatabase.sql 실행 시 자동 스키마 생성(federatedTwins)

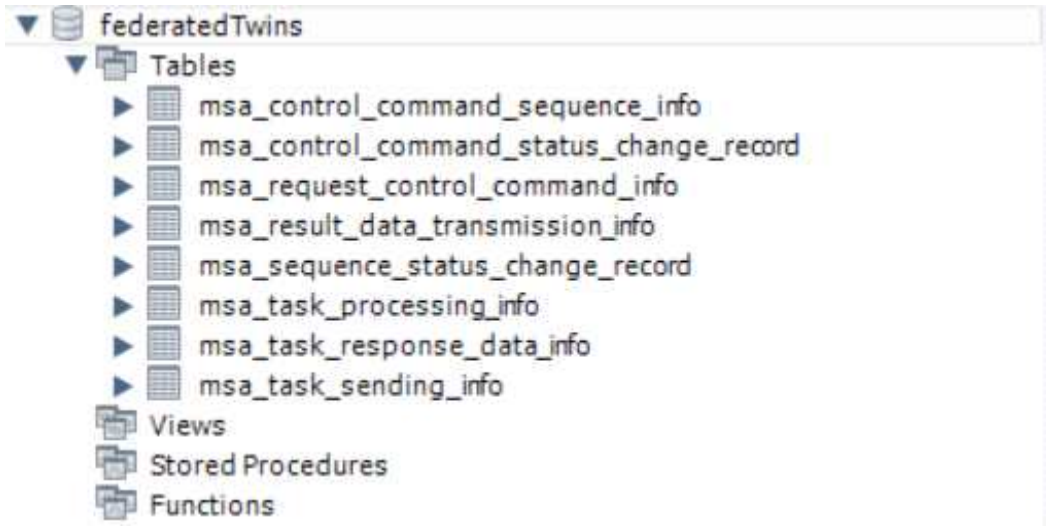
4) 기본 계정 정보 생성 - ID : fdtTwin / PW : Fdt1234!@#$

(계정 정보 수정 시 apprication.propertise에 jdbc 접속 정보 수정)



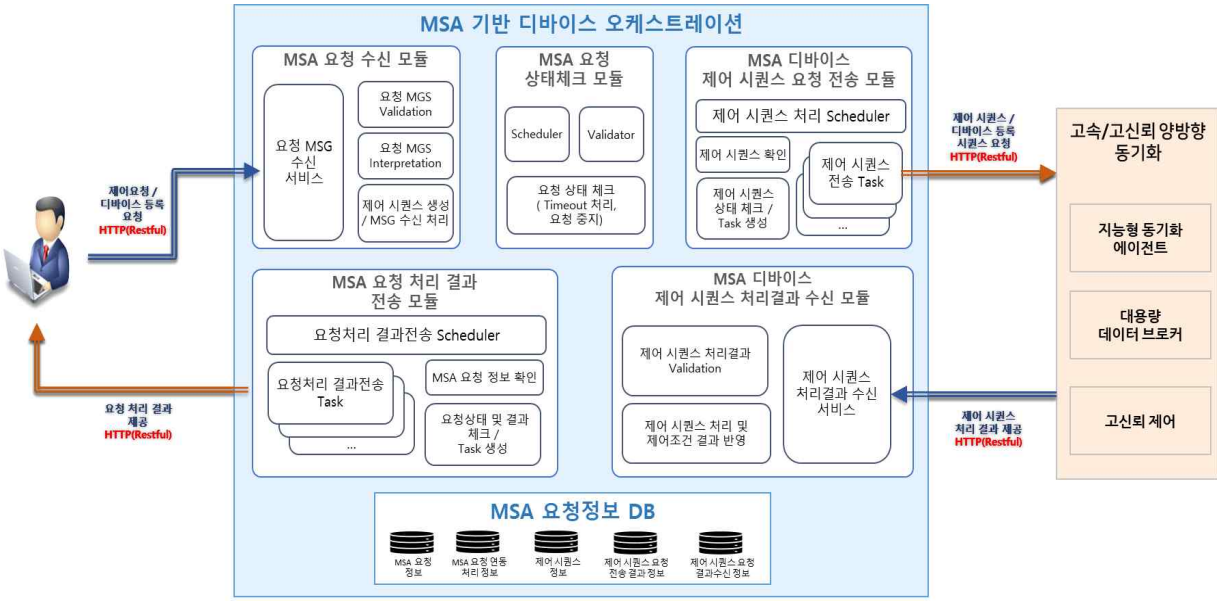
5) 파일 실행 후 프로젝트 하위에 있는 createDatabase.sql 파일은 삭제

6) 테이블 정보 확인



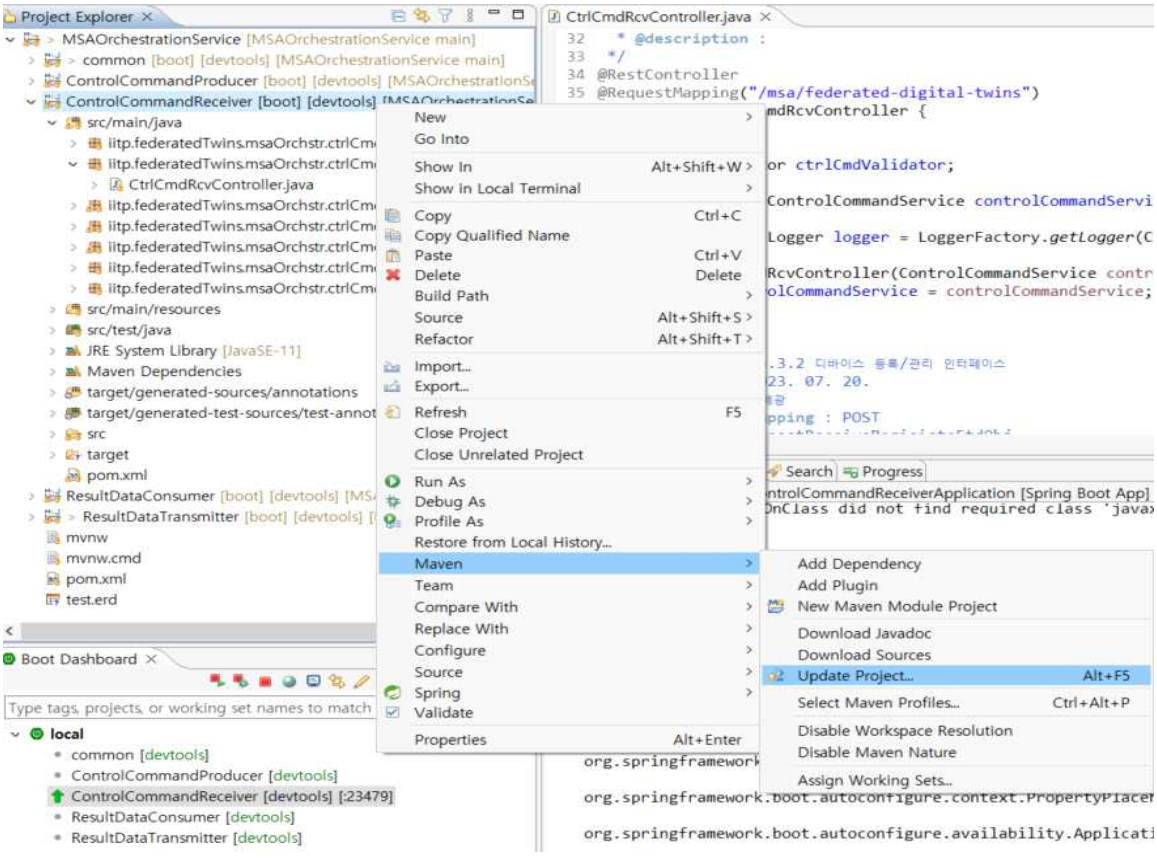
2. 구동/실행

가. MSA 기반 디바이스 오케스트레이션 S/W 구성

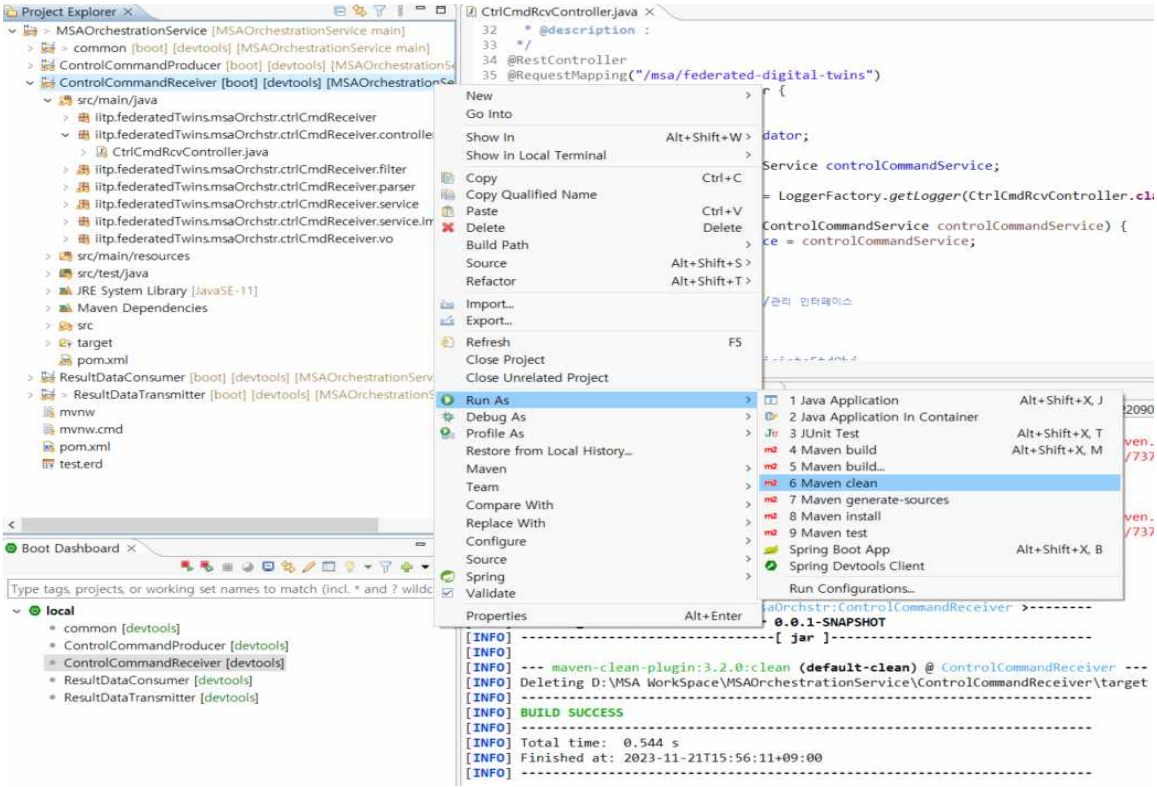


나. 프레임워크 구동/실행

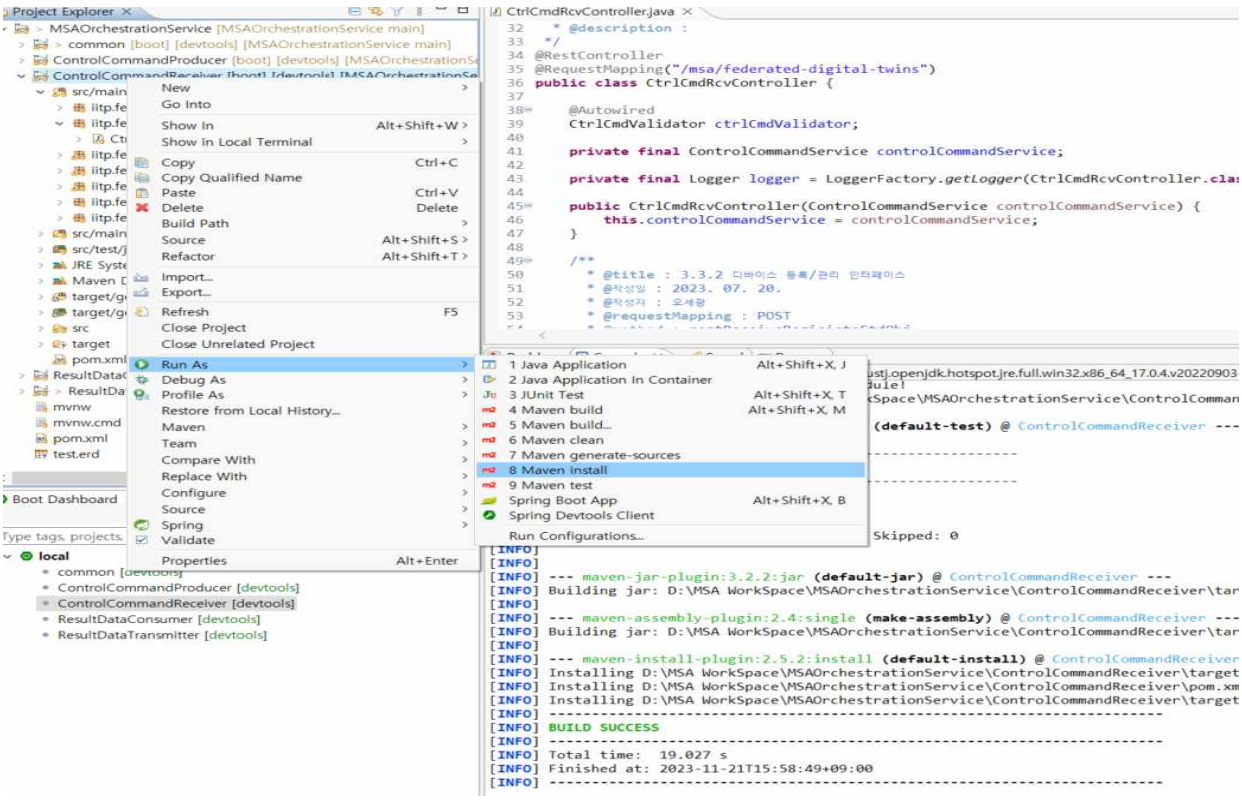
1) 모듈 프로젝트 우클릭 - Maven - Update Project



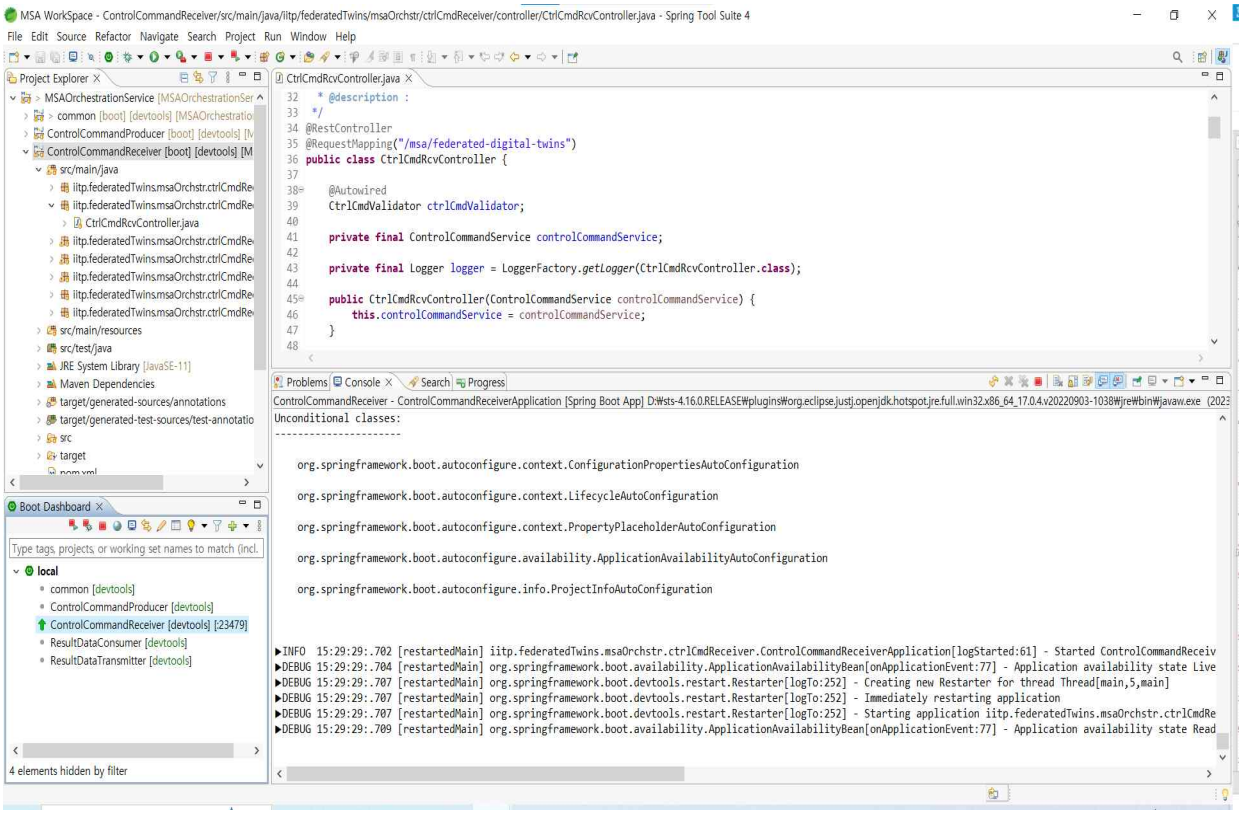
2) 모듈 프로젝트 우클릭 - Run As - Maven Clean



3) 모듈 프로젝트 우클릭 - Run As - Maven Install



4) Boot Dashboard - local - ControlCommandReceiver 모듈 우클릭 - start



3. 기능 설명

가. MSA기반 디바이스 오케스트레이션 기능 및 구성

1) 업무 프로세스별 기능 모듈 구성

(a) 업무 프로세스 구성

- Spring Boot Maven Multi Module 구조

(b) 업무 모듈 구성

・공통모듈 :Common

・업무 프로세스 모듈 : MSA 업무 공통 모듈(Common)
MSA 요청 수신모듈(ControlCommandReceiver)
MSA 제어시퀀스 요청 전송 모듈(ControlCommandProducer)
MSA 제어시퀀스 처리 결과 수신 모듈(ResultDataCunsumer)
MSA 요청처리 결과 전송 모듈(ResultDataTransMitter)
MSA 요청 상태체크 모듈(ControlCommandStatusChecker)

2) 업무 프로세스 모듈 상세

(a) MSA 업무 공통 모듈(Common)

・모듈 정의

- 각 모듈에서 사용하는 공통 기능을 관리하는 모듈로서 공통모듈에서 사용하는 기능은 확장 가능한 기능들로 설정되어 있음.

・ 모듈 기능

- DB Connection Pool(HikariCp), Mybatis 연동 Mapper, dto 및 Resource 설정
- 모듈에서 공통 사용가능 한 abstract class, interface 정의
- Sequence Task abstract Class 및 공통 정보처리 Service Interface 정의 및 구현
- 공통 사용 Util, Exception 등 Object 정의 및 구현
- 공통 Resource 정보 정의 및 설정

(b) MSA 요청 수신모듈(ControlCommandReceiver)

・모듈 정의

- 1, 3세부에서 전송하는 제어요청/디바이스 등록에 대한 요청정보를 수신하며 수신되는 제어명령을 파싱 및 유효성체크 및 처리 Task Indexing 작업을 진행

・ 모듈 기능

- Control Command Factory , Control Command Validator, 유효성 처리 Filter 기능 구현
- 제어명령 수신처리 및 Business Logic 기능 처리를 위한 controller, service, mapper, vo 기능 구현

(c) MSA제어시퀀스 요청 전송 모듈(ControlCommandProducer)

・모듈 정의

- 수신된 명령을 단위 시퀀스 단위로 분할하여 처리 시퀀스 단위로 처리 가능한 Task 정보 생성 및 고속/고신뢰 양방향 동기화 엔진에 실행 단위 제어시퀀스 요청(Http Restful) 처리

· 모듈 기능
- 제어명령 처리를 위한 Scheduler(요청 분배, Task 실행, Task 전송) 기능 구현
- Task 제어명령 처리 및 Business Logic 기능 처리를 위한 service, mapper, vo 기능 구현
- 실행 단위 시퀀스(단일 제어Task, 다중처리 Task) 요청 처리를 위한 Task process 객체 기능 및 고속/고신뢰 양방향 동기화 엔진에 제어시퀀스 정보를 요청하는 HttpRestful Task 구현

(d) MSA 제어시퀀스 처리 결과 수신 모듈(ResultDataCunsumer)
·모듈 정의
- 고속/고신뢰 양방향 동기화 엔진에 요청한 단위 제어시퀀스 처리 결과를 수신하는 모듈로서 수신되는 처리 결과확인(제어판단)/반영 및 다음 처리를 위한 결과 Filter적용 시퀀스 동작 반영 처리 기능 제공

· 모듈 기능
- 제어시퀀스 처리 결과 수신처리 및 Business Logic 기능 처리를 위한 controller, service, mapper, vo 기능 구현
- 유효성 처리를 위한 Filter 기능 구현

(e) MSA 요청처리 결과 전송 모듈(ResultDataTransMitter)
·모듈 정의
- 종료(정상, 세션 만료, 오류 등)된 제어 명령에 대한 요청정보 결과를 전송하는 모듈로서 요청정보 전송 시 CallBackURL 정보를 활용하여 처리결과 정보를 전송

· 모듈 기능
- 종료 처리된 명령의 결과 전송처리를 위한 Scheduler(전송 분배, 결과전송 실행Task) 기능 구현
- 결과전송 Task 처리 및 Business Logic 기능 처리를 위한 service, mapper, vo 기능 구현
- 결과 전송처리를 위한 Task process 객체 기능 및 결과 정보를 전송하는 HttpRestful Task 구현

(f) MSA 요청 상태체크 모듈(ControlCommandStatusChecker)
·모듈 정의
- 제어 명령 및 제어시퀀스에 대한 처리 상태(세션체크 및 처리가능 상태 확인 등)를 확인하는 모듈

· 모듈 기능
- 상태체크 처리를 위한 Scheduler 기능
- 제어 명령 확인 및 제어시퀀스 정보 확인처리 및 Business Logic 기능 처리를 위한 service, mapper, vo 기능 구현
- 실행 단위 시퀀스(단일 제어Task, 다중처리 Task) 처리 위한 Task process 객체 기능 구현